

1.1操作系统的基本概念



1.2操作系统的发展与分类



1.3操作系统的运行环境



注：仅供王道VIP学员使用 严禁外部传播！

1.4大内核与微内核

大内核

将操作系统的主要功能模块进行集中，从而用以提供高性能的系统服务

优点：各个管理模块之间共享信息，能够有效利用相互之间的有效特性，所有有着巨大的性能优势

缺点：层次交互关系复杂，层次接口难以定义，层次之间界限模糊

微内核

背景：随着计算机体系结构的不断发展，操作系统提供的服务越来越多,接口形式越来越复杂

将内核中最基本的功能（如：进程管理）保留在内核，将不需要在核心态执行的功能转移到用户态执行，降低内核设计的复杂性

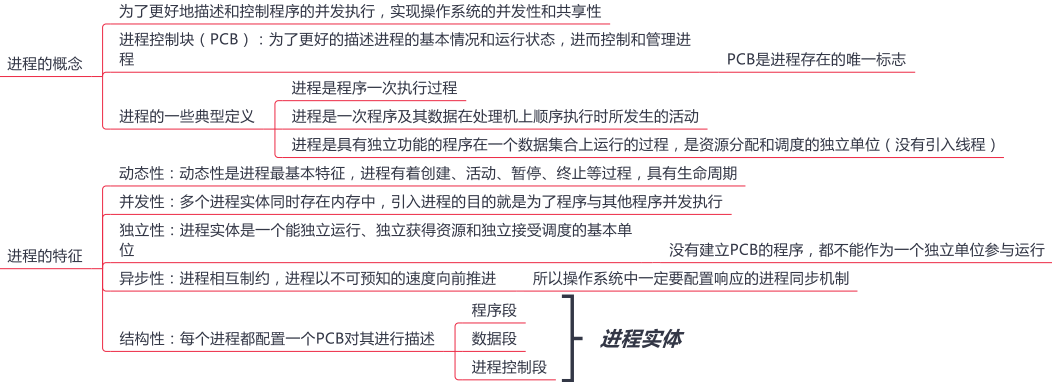
优点 有效的分离内核与服务、服务与服务、使得他们之间的接口更加的清晰，维护的代价大大降低

各部分可以独立的优化和演进

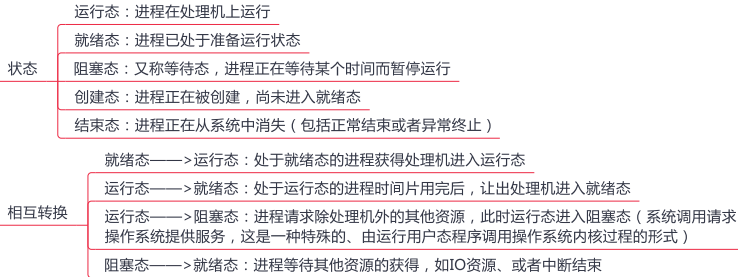
缺点 性能问题，需要频繁的在核心态和用户态之间进行切换

2.1进程与线程（上）

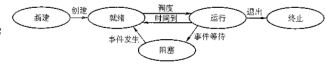
进程的概念和特征



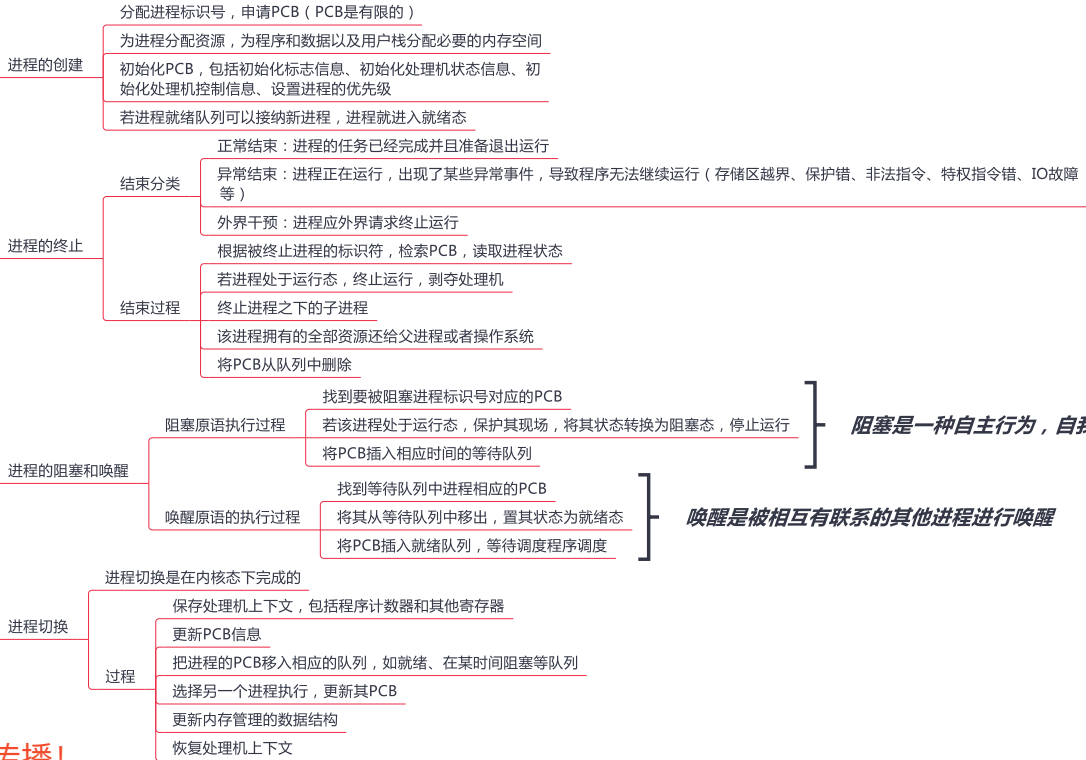
进程的状态与转换



相互转换



进程控制



唤醒是被相互有联系的其他进程进行唤醒

2.1进程与线程（中）

进程的组织

进程是一个独立的运行单位，也是操作系统进行资源分配和调度基本单位，由以下三部分构成

进程描述信息

进程标识符：标志进程

用户标识符：进程归属的用户，主要为共享和保护服务

进程当前状态：描述进程状态信息

进程优先级：描述进程抢战处理机优先级

代码运行入口地址

程序的外存地址

进入内存时间

处理机占用时间

信号量使用

进程控制和管理信息

用以说明有关内存地址空间或者虚拟地址空间状况，所打开的文件的列表和所使用的的输入/输出设备信息

代码段指针、数据段指针、堆栈段指针、文件描述符、键盘、鼠标

资源分配清单

处理机中各寄存器的值

通用寄存器值、地址寄存器值、控制寄存器值，标志寄存器值，状态字

处理机相关信息

程序段：能被进程调度程序调度到CPU执行的程序代码段

数据段：进程对应的程序加工处理的原始数据或者程序执行时产生的中间或者最终结果

进程的组织方式

链接方式

按照进程状态将PCB分为多个队列

操作系统持有指向各个队列的指针

索引方式

根据进程状态的不同,建立几张索引表

操作系统持有指向各个索引表的指针

进程的通信

共享存储

通信进程之间存在一块可以被直接访问的共享空间

低级方式：基于数据结构共享

高级方式：基于存储区共享

操作系统只负责为通信进程提供可共享使用的存储空间和同步互斥工具，数据交换则由用户自己安排读/写指令完成

消息传递

进程间的数据交换是以格式化的消息为单位的，进程通过系统提供的发送消息和接收消息的两个原语进行数据交换

直接通信方式：发送进程直接发送消息给接收进程，并将它挂在接收进程的消息缓冲队列上，接收进程从消息缓冲队列中取得消息

间接通信方式：发送进程把消息发送给某个中间实体，接收进程从中间实体中获得消息 例如电子邮件系统

管道通信

发送进程以字符流形式将大量数据送入写管道，接收进程从管道中接收数据

当管道写满时，写进程的write()系统调用将被阻塞，等待读进程将数据取走

当读进程将数据全部取走后，管道变空，此时读进程的read()系统调用将被阻塞

功能：互斥、同步、确定对方存在

半双工通信，不可以同时读和写

限制管道的大小

管道变空的时候阻塞读进程

管道中的数据被读取后会马上被丢弃

线程概念和多线程模型

线程基本概念

减小程序在并发执行时所付出的时空开销，提高操作系统的并发性能

引入线程后，进程只作为系统资源的分配单元，线程作为处理机的分配单元

线程与进程的比较

传统中进程是资源和独立调度的基本单位

引入线程后，进程是独立调度的基本单位，线程是资源的基本单位

不同进程的线程切换会引起进程切换

拥有资源 进程是资源分配的基本单位

并发性 引入线程后，进程可以并发执行，多个线程之间也可以并发执行，提高了系统的吞吐量

系统开销 同一进程的线程切换要比进程切换开销小的多

地址空间和其他资源 进程的地址空间之间相互独立，统一进程的各线程之间共享进程的资源，某进程的线程对其他进程不可见

通信方面 进程间通信需要进程同步和互斥手段的辅助，保证数据的一致性

线程间可以直接读/写进程程序段来进行通信

线程属性

不拥有系统资源，拥有唯一标识符和线程控制块

不同的线程可以执行相同的程序，同一个服务程序被不同用户调用时，操作系统将其创建为不同线程

统一进程的线程共享该进程拥有的全部资源

线程是处理机的独立调度单位

线程也有生命周期，阻塞，就绪，运行等状态

多CPU计算机中,各个线程可占用不同的CPU

每个线程都有一个线程ID、线程控制块（TCB）

切换同进程内的线程,系统开销很小

切换进程,系统开销较大

由于共享内存地址空间,同一进程中的线程间通信甚至无需系统干预

注：仅供王道VIP学员使用 严禁外部传播！

2.1进程与线程（下）

线程的实现方式

- 用户级线程 有关线程管理的所有工作都由应用程序完成，内核意识不到线程的存在
- 内核级线程 线程的管理工作全部由内核完成

多线程模型

- 多对一
 - 经多个用户级线程映射到一个内核级线程，线程管理在用户空间完成，用户级线程对操作系统不可见
 - 优点：线程管理是在用户控件进行的，效率比较高
 - 缺点：一个线程阻塞全部线程都会阻塞，多个线程不能并行运行在多处理机上
- 一对一
 - 每个用户级线程映射到一个内核级线程上
 - 优点：并发能力强
 - 缺点：创建线程开销大，影响应用程序的性能
- 多对多
 - 多个线程映射到多个内核线程上
 - 结合上述两种，既可以提高并发性，又适当的降低了开销

2.2处理机调度（上）



2.2处理机调度（下）

典型调度算法



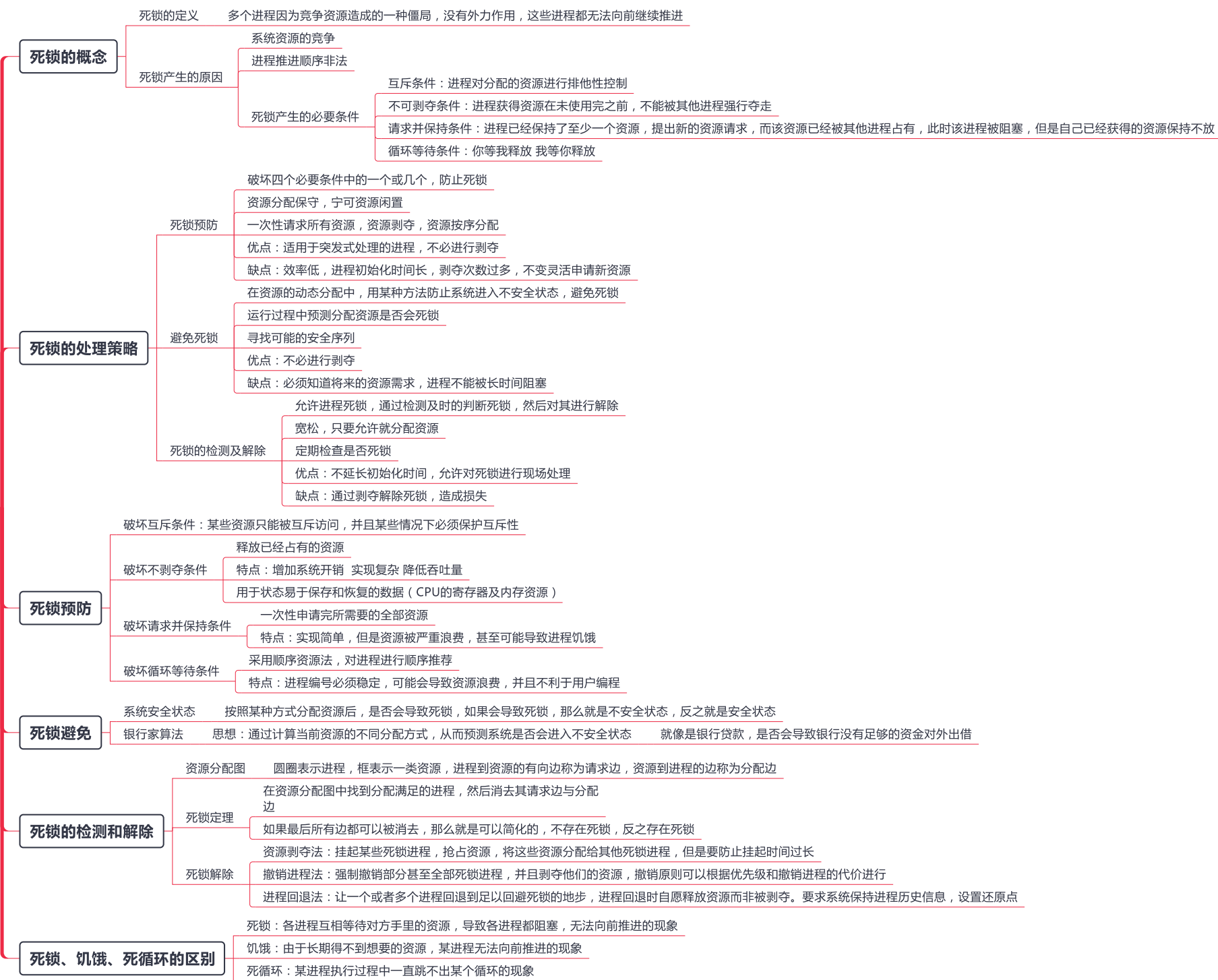
注：仅供王道VIP学员使用 严禁外部传播！

2.3进程同步



注：仅供王道VIP学员使用 严禁外部传播！

2.4 死锁



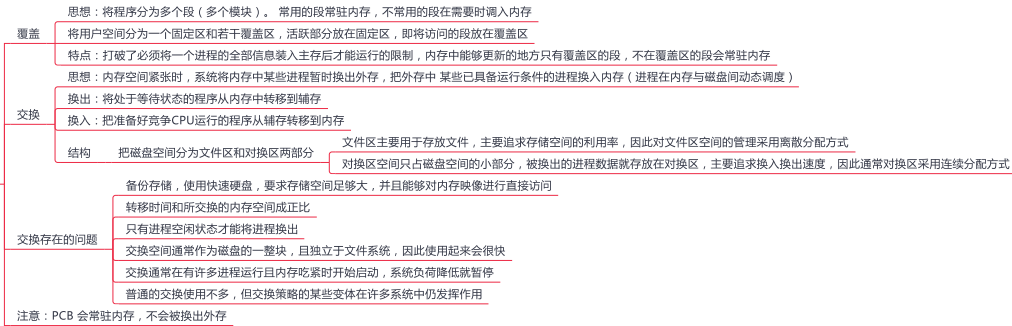
注：仅供王道VIP学员使用 严禁外部传播！

3.1 内存管理概念（上）

内存管理的基本原理和要求



覆盖与交换



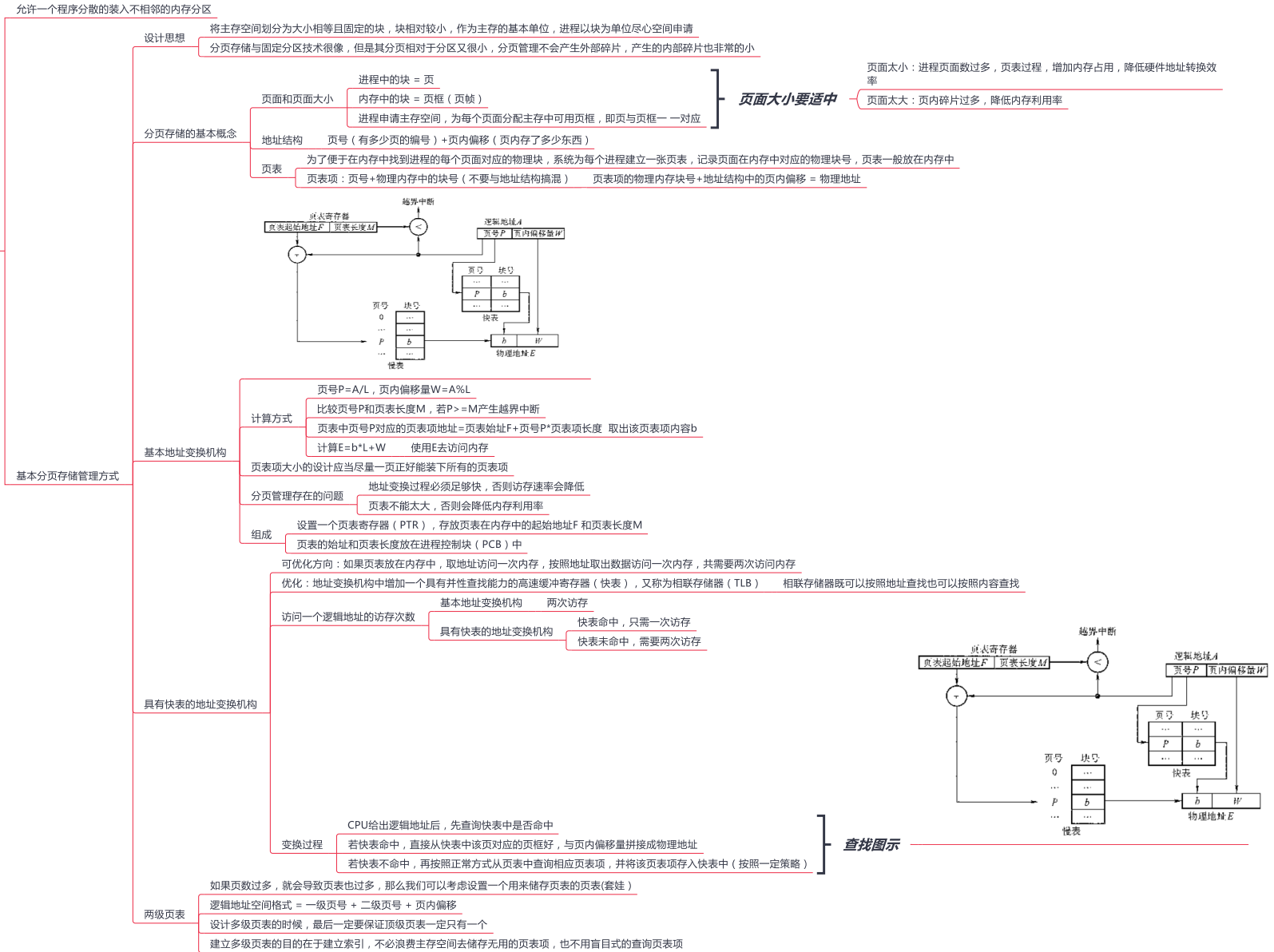
连续分配管理方式



注：仅供王道VIP学员使用严禁外部传播！

3.1 内存管理概念（中）

非连续分配管理方式



注：仅供王道VIP学员使用严禁外部传播！

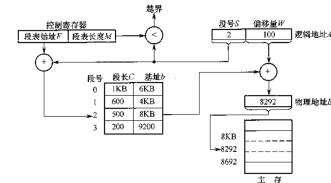
3.1 内存管理概念（下）

基本分段存储管理方式

- 出发点
 - 分页是从计算机角度考虑设计的，目的是为了内存的利用率，提高计算机性能，分页通过硬件机制实现，对用户完全透明
 - 分段是从用户和程序员的角度提出，满足方便编程，信息保护和共享，动态增长及动态链接等多方面的需要
- 分段
 - 按照用户进程中的自然段划分逻辑空间
 - 地址结构 = 段号S + 段内偏移量W
- 段表
 - 页式系统中，页号和页内偏移对用户透明 段式系统中 段号和段内偏移量必须由用户显示的提供
 - 每个进程都有一张逻辑空间与内存空间映射的段表，这个段表项对应进程的一段，段表项记录该段在内存中的起始地址和长度
 - 段表内容 = 段号 + 段长 + 本段在内存中的地址

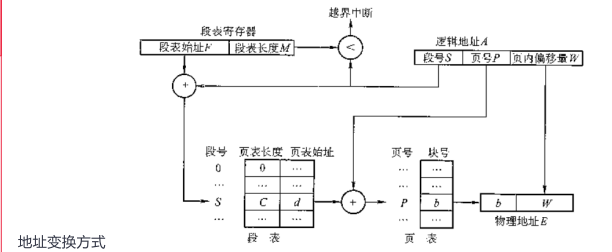
- 地址变换机构
 - 逻辑地址A中取出段号S和段内偏移量W
 - 比较段号S和段表长度M，若S>=M,则产生越界中断，否则继续执行
 - 段号S对应的段表项地址 = 段表起始地址F+段号S*段表项长度，从该段表项中取出段长C，比较段内偏移量与C的大小判断是否出现越界
 - 取出段表项中该段的起始地址b,计算E = b + W，用得到的物理地址E去访问内存
- 段的共享与保护
 - 共享：两个作业的段表中响应表项指向被共享段的同一个物理副本来实现的 纯代码或者可重入代码以及不可修改的数据可以被共享
 - 保护机制
 - 存取控制保护
 - 地址越界保护

变换过程



段页式管理方式

- 页式存储有效的提高内存利用率，分段存储能反映程序的逻辑结构并有利于段的分享，将这两种方式结合一下 这种二者结合的方法经常在计算机理论中遇到
- 思想
 - 作业的地址空间首先被分成若干逻辑段，每段有自己的段号
 - 每个段分成若干大小固定的页
 - 对内存空间的管理仍然和分页存储管理一样
- 地址结构 段号S + 页号P + 页内偏移量W
- 为了实现地址变换，系统为每个进程建立了一张段表，每个分段有一个页表 一个进程中，段表只能有一个，页表可以有多个



补充

- 不能被修改的代码称为纯代码或可重入代码（不属于临界资源）
- 分段与分页的区别
 - 分页对用户不可见，分段对用户可见
 - 分页的地址空间是一维的，分段的地址空间是二维的
 - 分页（单级页表）、分段访问一个逻辑地址都需要两次访存，分段存储中也可以引入快表机构
 - 分段更容易实现信息的共享和保护（纯代码可重入代码可以共享）
- 分段与分页优缺点
 - 分页管理
 - 优点：内存空间利用率高，不会产生外部碎片，只有少量的页内碎片
 - 缺点：不方便按照逻辑模块实现信息的共享和保护
 - 分段管理
 - 优点：很方便按照逻辑模块实现信息的共享和保护
 - 缺点：如果段长过大，为其分配很大的连续空间会很不方便
 - 段式管理会产生外部碎片

3.2虚拟内存管理（上）



3.2虚拟内存管理（下）

页面置换算法

- 最佳置换算法（OPT）
 - 选择永不使用或者最长时间内不再访问的页面进行淘汰，但是现实中是无法预知的
 - 优点：缺页率最小，性能最好
- 先进先出页面置换算法（FIFO）
 - 优先淘汰最早进入的页面
 - 优点：实现简单
 - 缺点：与进程的实际运行规律不匹配
 - Belady异常：增大分配的物理块数但是故障数不减反增 只有先进先出算法会出现
- 最近最久未使用（LRU）置换算法
 - 选择最近最长时间内没有被访问的页面进行淘汰，每个页面设置一个访问字段，用来标识上次被访问到现在经历的时间
 - 优点：性能好
 - 缺点：实现复杂 需要寄存器和栈的硬件支持 LRU是堆栈类算法
- 时钟(CLOCK)置换算法
 - 像一个时钟一样转圈，每个页面设置一个使用位（访问位），遇到没有被使用的就会将页面换出，然后将使用位置0，如果遇到使用的就会将使用位置零，然后扫描下一个
 - 优点：性能接近于最佳置换算法
 - 缺点：实现复杂 开销大
- 改进型CLOCK算法
 - 使用位（访问位）的基础上增加修改位
 - 扫描过程
 - 扫描缓冲区，选择第一个使用位和修改位都为0的页面换出
 - 第一步失败后，查找使用位为0，修改位为1的进行替换，对于每个跳过的帧，将使用位置为0
 - 第二步失败后，指针回到初始地点且使用位（访问位）均为0，重复第一步
 - 优点：相对于未改进型，节省了时间

页面分配策略

- 驻留集：给一个进程的分配的物理页框的集合就是这个进程的驻留集
- 考虑因素
 - 分配给一个进程的的存储量越小，任何时候驻留在主存中的进程数就越多，可以提高处理机的时间利用率
 - 一个进程在主存中的页数过少，页错误率就会相对较高
 - 页数过多，对进程的错误率也不会产生过多的影响
- 分配策略
 - 固定分配局部置换
 - 每个进程分配固定物理块数，缺页的时候就进行换页
 - 难以确定每个进程应该分配的物理块数
 - 太多导致资源利用率下降 太少导致频繁缺页中断
 - 进程分配一定物理块，系统自身保留一定空闲物理块，如果进程缺页，就对该进程分配新的物理块
 - 可变分配全局置换
 - 优点：最容易实现，动态调整物理块分配
 - 缺点：如果盲目分配物理块，就会导致多道程序并发能力下降
 - 根据进程的缺页情况，对物理块进行动态分配，如果频繁缺页，就对其多分配物理块，如果缺页率特别低，就减少其物理块
 - 可变分配局部置换
 - 优点：保持了系统的多道程序并发能力
 - 缺点：增大了开销，实现复杂
- 调入页面的时机
 - 预调页策略
 - 将预计不久被访问的页面调入，成功率约为50%
 - 请求调页策略
 - 当进程提出缺页的时候，再按照一定策略进行调页
 - 特点：一次调入一页，调入/调出页面数多时会花费过多的I/O开销
- 从何处调入页
 - 拥有足够的对换空间 可以全部从对换区调入所需页面，提高调页速度
 - 缺少足够的对换区空间 不会被修改的文件从文件区调入，可能被修改的部分换入对换区，以后再从对换区调入 原理：读速度比写速度快
 - UNIX方式 进程相关文件访问文件区，没有运行的页面从文件区调入，曾经运行过但又被换出的页面放在对换区

一般情况下两种策略同时使用

抖动

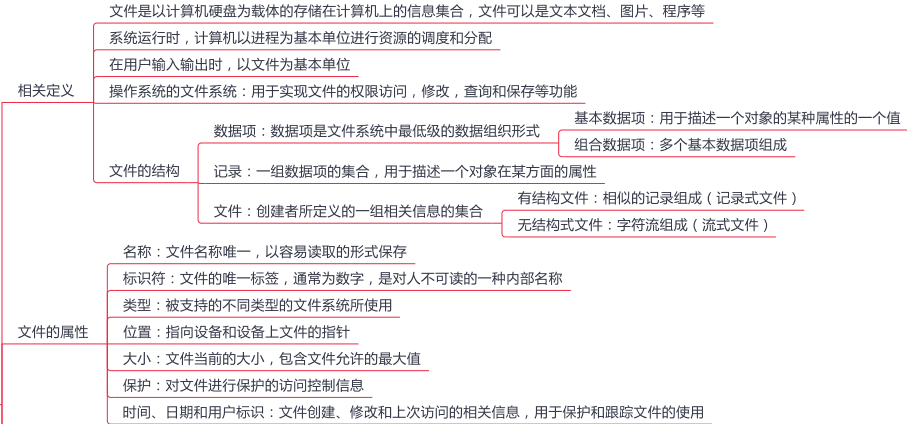
- 原因
 - 刚换出的页面又要换入内存
 - 分配的物理页帧数不足（主要原因）
 - 置换算法不当

工作集

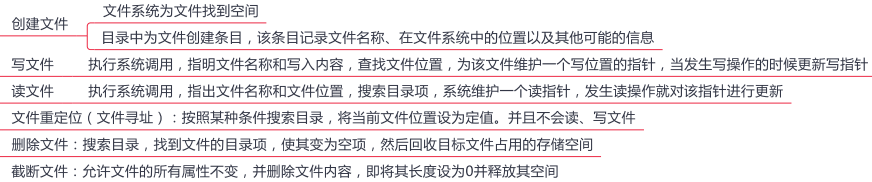
- 某段时间内，进程要访问的页面集合。
- 原理
 - 操作系统跟走每个进程的工作集，并为进程分配大于其工作集的物理块
 - 落入工作集的页面需要调入驻留集中，落在工作集外面的页面可以从驻留集中换出
 - 若还有空闲物理块，可以再调入一个进程到内存以增加多道程序数。
 - 若所有进程的工作集之和超过了可用物理块的总数，操作系统就会暂停一个进程，并将其页面调出并将其物理块分配给其他进程

4.1文件系统基础（上）

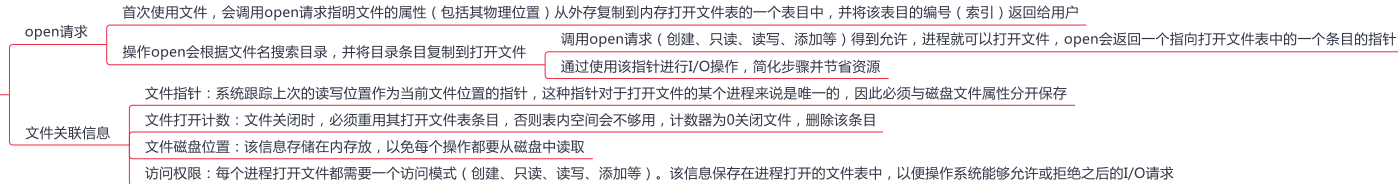
文件的相关概念



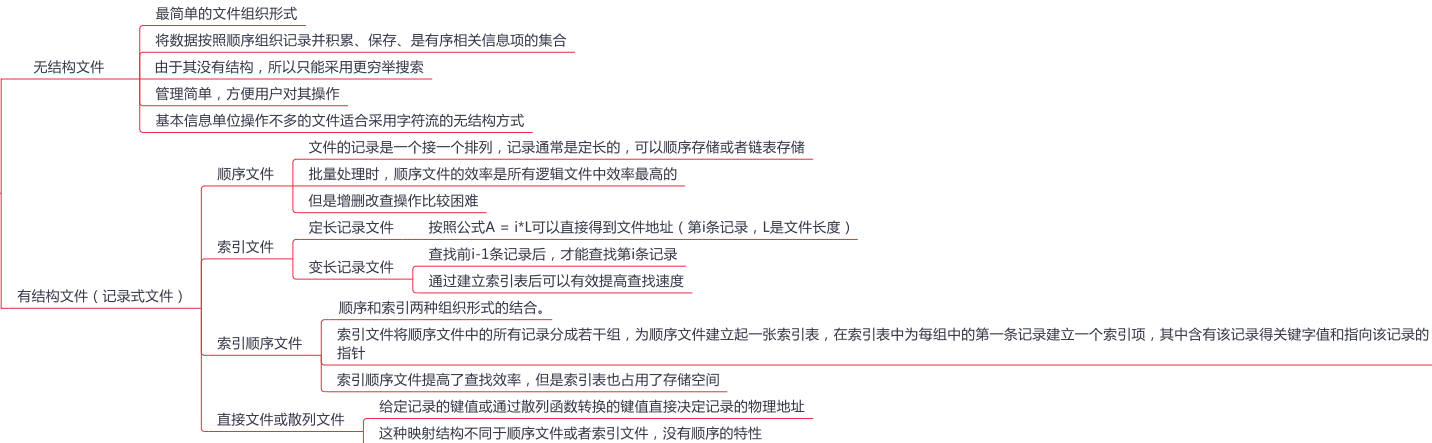
文件的基本操作



文件的打开与关闭



文件的逻辑结构



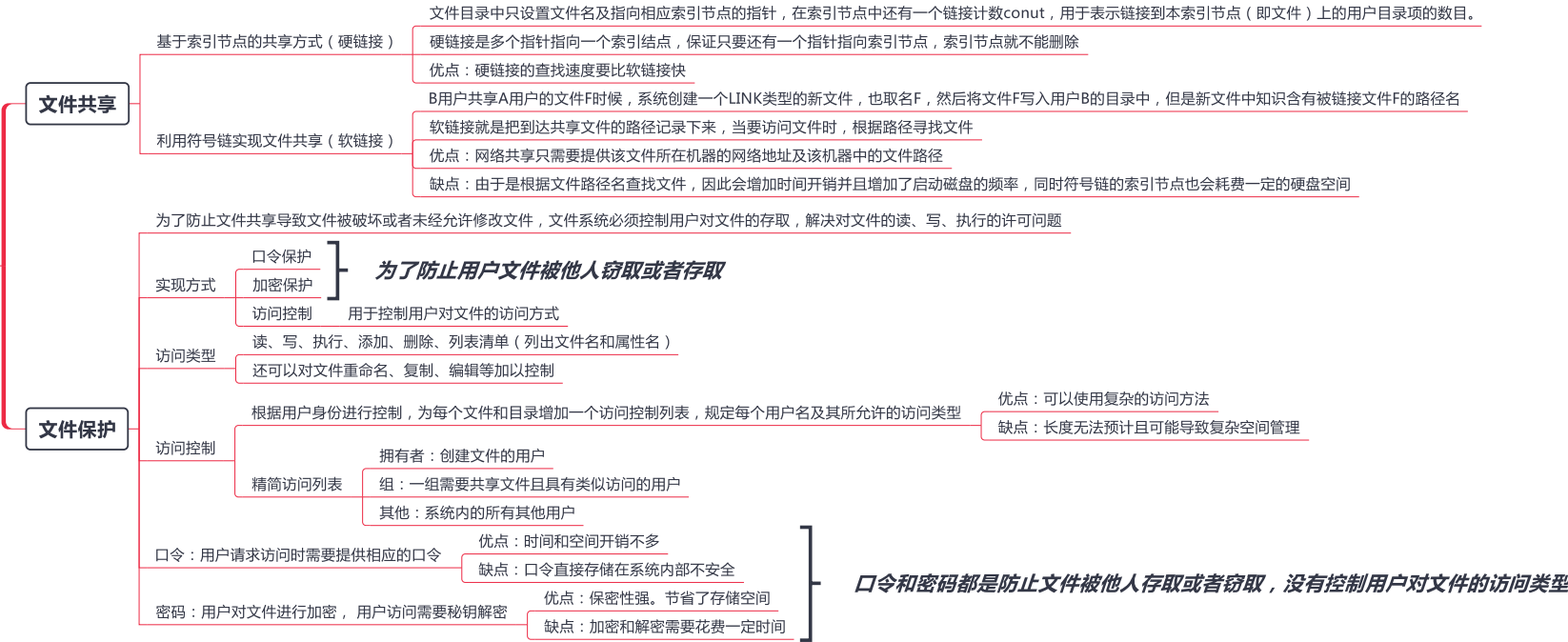
注：仅供王道VIP学员使用严禁外部传播！

4.1文件系统基础（中）

目录结构



4.1文件系统基础（下）



4.2文件系统的实现

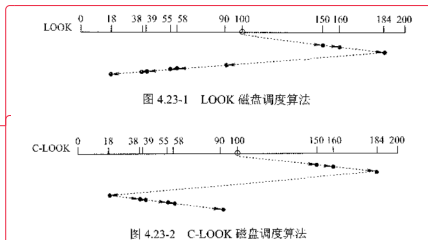


4.3 磁盘组织与管理

磁盘结构

- 表面涂有磁性物质的金属或塑料构成的圆形盘片，通过一个称为磁头的导体线圈从磁盘读取数据。
- 磁盘盘面上的数据存储在在一组同心圆中，称为磁道
- 一个盘面上有上千个磁道，磁道又划分为几百个扇区，每个扇区固定存储大小，一份扇区称为一个盘块
- 磁盘地址用 柱面号-盘面号-扇区号(块号) 表示
- 磁盘分类
 - 固定头磁盘：磁头相对于盘片的径向方向固定
 - 活动头磁盘：每个磁道一个磁头，磁头可以移动
 - 固定盘磁盘：磁头臂可以来回伸缩定位磁道，磁盘永久固定在磁盘驱动器内
 - 可换盘磁盘：可以移动和替换

磁盘调度算法

- 读写时间组成
 - 寻找时间：活动头磁盘在读写信息前，将磁头移动到指定磁道所需要的时间（跨越n条磁道+启动磁臂）： $T_s = m \cdot n + s$
 - 延迟时间：磁头定位到某一磁道扇区所需要的时间， $T_r = 1/2r$ （转速为r）
 - 传输时间：从磁盘读出或向磁盘写入数据经过时间， $T_r = b / rN$
- 磁盘调度算法
 - 先来先服务算法（FCFS）
 - 按照进程请求访问磁盘的先后顺序进行调度
 - 优点：公平 实现简单
 - 缺点：适用于少量进程访问，如果进程过多算法更倾向于随机调度
 - 最短寻找时间有限算法（SSTF）
 - 选择调度处理的磁道是与当前磁头所在磁道距离最近的磁道
 - 优点：性能强于先来先服务算法
 - 缺点：容易产生饥饿现象
 - 扫描(SCAN)算法
 - 在磁头当前移动方向上选择与当前磁头所在的磁道距离最近的请求作为下一次服务对象
 - 优点：寻道性能好，可以避免饥饿现象
 - 缺点：对最近扫描过的区域不公平，访问局部性方面不如FCFS和SSTF好
 - 循环扫描算法（C-SCAN）
 - 磁头单向移动，回返时直接回到起始端，而不服任何请求
 - LOOK与C-LOOK算法
 - 在SCAN与C-SCAN算法的基础上规定了查看移动方向上是否有请求，如果没有就不会继续向前移动，而是直接改变方向（LOOK）或者直接回到第一个请求处（C-LOOK）
 - 
 - 对盘面交替编号 原因：磁头在读/写一个物理块后，需要经过短暂的处理时间才能开始处理下一块

磁盘的管理

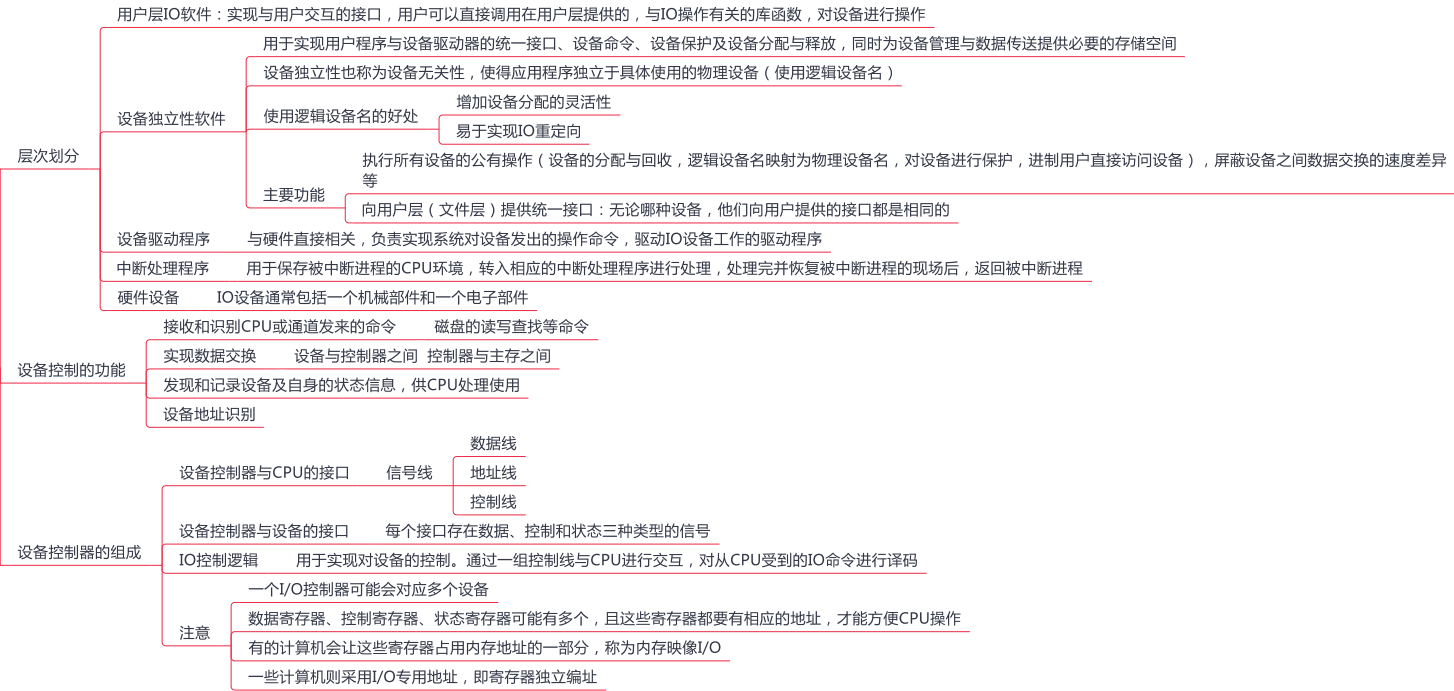
- 磁盘初始化
 - 低级格式化：磁盘分扇区，为每个扇区采用特别的数据结构（头、数据区域、尾部组成），头部含有一些磁盘控制器所使用的信息
 - 进一步格式化处理：磁盘分区，对物理分区进行逻辑格式化（创建文件管理系统），包括空闲和已分配的空间及一个初始为空的目录
- 引导块
 - 计算机启动时运行自举程序，初始化CPU寄存器、设备控制器和内存等，然后启动操作系统
 - 组局程序通常保存在ROM中，在ROM中保留很小的自举块，完整的自举程序保存在启动块上
 - 拥有启动分区的磁盘称为启动磁盘或系统磁盘
- 坏块
 - 无法使用的扇区
 - 对于简单的磁盘，可以在逻辑格式化时（建立文件系统时）对整个磁盘进行坏块检查，标明哪些扇区是坏扇区，比如：在 FAT 表上标明
- 处理方式
 - 简单磁盘：手动处理，对坏块进行标记，程序不会使用
 - 复杂磁盘：控制器维护一个磁盘坏块链表，同时将一些块作为备用，用于替代坏块（扇区备用）

5.1 IO管理概述（上）



5.1 IO管理概述（下）

I/O子系统的层次结构



5.2 IO核心子系统（上）



5.2 IO核心子系统（下）

IO调度

- 设备分类
 - 独占设备——一个时段只能分配给一个进程（如打印机）
 - 共享设备——可同时分配给多个进程使用（如磁盘），各进程往往是宏观上同时共享使用设备，而微观上交替使用
 - 虚拟设备——采用 SPOOLing 技术将独占设备改造成虚拟的共享设备，可同时分配给多个进程使用（如采用 SPOOLing 技术实现的共享打印机）

- 设备分配的安全性
 - 安全分配方式
 - 进程发出IO请求后便进入阻塞态，直到IO结束才被唤醒
 - 优点：设备分配安全
 - 缺点：CPU和IO设备串行工作
 - 不安全分配方式
 - 进程发出IO请求后继续运行，需要时发出第二个，第三个IO请求
 - 优点：进程推进迅速
 - 缺点：可能产生死锁

- 逻辑设备名到物理设备名的映射
 - 目的
 - 提高设备分配的灵活性和利用率
 - 实现IO重定向
 - 引入设备独立性
 - 实现方法：引入逻辑设备表(LUT)，用来将逻辑设备名映射为物理设备名
 - 建立方式
 - 整个系统设置一张LUT，所有设备分配情况都记录在这张表上（单用户设备）
 - 每个用户建立一张LUT，分别记录设备的分配情况

SPOOLING技术（假脱机技术）

- 目的
 - 缓解CPU 与IO 的速度差异矛盾
- 要实现SPOOLing 技术，必须要有多个道程序技术的支持
- 输入井和输出井
 - 输入井用来收容IO设备的数据
 - 输出井用来模拟输出时的磁盘
- 输入缓冲区和输出缓冲区
 - 输入缓冲区：暂存由输入设备送来的数据
 - 输出缓冲区：暂存从输出井送来的设备
- 输入进程和输出进程
 - 输入进程：模拟脱机输入时的外围控制机，将用户要求的数据从输入机通过输入缓冲区送到输入井中，当CPU需要数据，直接将输出井中的数据送入内存
 - 输出进程：模拟脱机输出时的外围控制机，把用户要求输出的数据先从内存送到输出井中，待输出设备空闲时，再将输出井中的数据经过输出缓冲区送到输出设备
- 特点
 - 提高了IO速度
 - 独占设备变成了共享设备
 - 实现了虚拟设备功能
- 通俗一点就是，如果设备被占用，我们就先把数据暂存一下，等到设备空闲了就把这些数据输送到设备中

高速缓存与缓冲区对比

